Socket basics

The Talo Socket provides an easy way for you to add interactivity to your game. Using sockets you can send messages between players and allow your game client to be reactive to changes (e.g. new channel messages).

Message structure

The socket used by Talo follows the web socket standard and is compatible with libraries that also do the same.

One major difference is that the Talo Socket will validate messages against a request type and payload structure. For example, if you send a message with the `v1.players.identified` request, your payload's data must also match the expected schema. The list of available requests and responses are available on their respective pages.

Using the identification example, here is the JSON expected by the Talo Socket when identifying a player:

```
{
  "req": "v1.players.identified",
  "data": {
    "playerAliasId": 105,
    "socketToken": "9aae3c26-98d0-4a13-9dd7-b187aa36f6cd"
  }
}
```

The request type is defined by the `req` key and the payload of the request is defined using the `data` key.

Responses use a similar message structure, except that `req` is replaced by `res` to denote a response:

```
{
  "res": "v1.players.identified.success",
  "data": {
    "id": 105,
    "service": "username",
    "identifier": "billy",
    "player": {
      "id": "85d67584-1346-4fad-a17f-fd7bd6c85364",
      "props": [],
      "devBuild": false,
```

```
      "createdAt": "2024-10-25T18:18:28.000Z",
      "lastSeenAt": "2024-12-04T07:15:13.000Z",
      "groups": []
    },
    "lastSeenAt": "2024-12-04T07:15:13.000Z",
    "createdAt": "2024-10-25T18:18:28.000Z",
    "updatedAt": "2024-12-04T07:15:13.000Z"
  }
}
```


Authentication

The Talo Socket provides two layers of authentication:

1. You must provide a socket ticket when opening a socket connection. This identifies your game and the key's scopes are used to determine whether a client is eligible to receive specific responses.
2. When players are identified, a `socketToken` is stored against the player alias for 1 hour. While the socket token is valid, you can use it to authenticate your player with the socket.

> **INFO**
>
> Not all requests and responses will need a player to be identified. However, your access key will need the `read:players` scope to identify players.

> **WARNING**
>
> Unlike the HTTP API, players cannot be created during identification. To successfully identify a player, their alias must already exist in Talo.

Connecting to the socket server

The default socket URL is `wss://api.trytalo.com`. When opening a socket connection, you must provide a socket ticket as a query parameter. The full URL should look like this:

```
wss://api.trytalo.com/socket?ticket=your-ticket-here
```

Your ticket should be a valid socket ticket, i.e. it was created within the last 5 minutes. Visit the API docs for more information on how to obtain a socket ticket.

Error responses

Due to the nature of sockets, error responses are completely asynchronous. Error responses follow a standard format so you can match them up against your requests. For example:

```
{
  "res": "v1.error",
  "data": {
    "req": "v1.players.identify",
    "message": "Invalid socket token",
    "errorCode": "INVALID_SOCKET_TOKEN"
  }
}
```

Some errors may also return a "cause" key which drills down into why a request failed:

```
{
  "res": "v1.error",
  "data": {
    "req": "v1.channels.message",
    "message": "An error occurred while processing the message",
    "errorCode": "LISTENER_ERROR",
    "cause": "Player not in channel"
  }
}
```

For a more detailed look into the available error codes, visit the common errors page.